

1 Introduction to axiomatic semantics

Now we turn to the third and final main style of semantics, *axiomatic semantics*. The idea in axiomatic semantics is to define meaning in terms of logical specifications that programs satisfy. This is in contrast to operational models (which show *how* programs execute) and denotational models (which show *what* programs compute). This approach to reasoning about programs and expressing program semantics was originally proposed by Floyd and Hoare and was developed further by Dijkstra and Gries.

A common way to express program specifications is in terms of pre-conditions and post-conditions:

$$\{Pre\} c \{Post\}$$

where c is a program, and Pre and $Post$ are formulas that describe properties of the program state, usually referred to as *assertions*. Such a triple is usually referred to as a *partial correctness specification* (or sometimes a “Hoare triple”) and has the following meaning:

“If Pre holds before executing c , and c terminates, then $Post$ holds after c .”

In other words, if we start with a store σ in which Pre holds and the execution of c with respect to σ terminates and yields a store σ' , then $Post$ holds in store σ' .

Pre-conditions and post-conditions can be regarded as interfaces or contracts between the program and its clients. They help users to understand what the program is supposed to yield without needing to understand how the program executes. Typically, programmers write them as comments for procedures and functions as documentation and to make it easier to maintain programs. Such specifications are especially useful for library functions for which the source code is often not available to the users. In this case, pre-conditions and post-conditions serve as contracts between the library developers and users of the library.

However, there is no guarantee that pre-conditions and post-conditions written informally in comments are correct: the comments describe the intent of the developer, but they do not give a guarantee of correctness. Axiomatic semantics addresses this problem. It shows how to rigorously describe partial correctness statements and how to establish correctness using formal reasoning.

Note that partial correctness specifications don’t ensure that the program will terminate—this is why they are called “partial”. In contrast, *total correctness statements* ensure that the program terminates whenever the precondition holds. Such statements are denoted using square brackets:

$$[Pre] c [Post]$$

meaning:

“If Pre holds before c then c will terminate and $Post$ will hold after c .”

In general a pre-condition specifies what the program expects before execution and the post-conditions specifies what guarantees the program provides (if the program terminates). Here is a simple example:

$$\{\text{foo} = 0 \wedge \text{bar} = i\} \text{ baz} := 0; \text{ while } \text{foo} \neq \text{bar} \text{ do } (\text{baz} := \text{baz} - 2; \text{foo} := \text{foo} + 1) \{\text{baz} = -2i\}$$

It says that if the store maps `foo` to 0 and `bar` to i before execution, then, if the program terminates, the final store will map `baz` to $-2i$ (i.e., -2 times the initial value of `bar`). Note that i is a logical variable that doesn't occur in the program and is only used to express the initial value of `bar`. Such variables are sometimes called *ghost variables*.

This partial correctness statement is valid. That is, it is indeed the case that if we have any store σ such that $\sigma(\text{foo}) = 0$, and

$$C \llbracket \text{baz} := 0; \text{ while } \text{foo} \neq \text{bar} \text{ do } (\text{baz} := \text{baz} - 2; \text{foo} := \text{foo} + 1) \rrbracket \sigma = \sigma',$$

then $\sigma'(\text{baz}) = -2\sigma(\text{bar})$.

Note that this is a *partial* correctness statement: if the pre-condition is true before c , and c **terminates** then the post-condition holds after c . There are some initial stores for which the program will not terminate.

The following total correctness statement is true.

$$[\text{foo} = 0 \wedge \text{bar} = i \wedge i \geq 0] \text{ baz} := 0; \text{ while } \text{foo} \neq \text{bar} \text{ do } (\text{baz} := \text{baz} - 2; \text{foo} := \text{foo} + 1) [\text{baz} = -2i]$$

That is, if we start with a store σ that maps `foo` to 0 and `bar` to a non-negative integer, then the execution of the command will terminate in a final store σ' with $\sigma'(\text{baz}) = -2\sigma(\text{bar})$.

The following partial correctness statement is not valid. (Why not?)

$$\{\text{foo} = 0 \wedge \text{bar} = i\} \text{ baz} := 0; \text{ while } \text{foo} \neq \text{bar} \text{ do } (\text{baz} := \text{baz} + \text{foo}; \text{foo} := \text{foo} + 1) \{\text{baz} = i\}$$

In the rest of our discussion of axiomatic semantics we will focus almost exclusively on partial correctness assertions.

2 Assertions

Now we turn to the following issues:

- What logic do we use for writing assertions? That is, what can we express in pre-conditions and post-condition?
- What does it mean that an assertion is valid? What does it mean that a partial correctness statement $\{Pre\} c \{Post\}$ is valid?
- How can we prove that a partial correctness statement is valid?

What can we say in pre-conditions and post-conditions? In the examples so far, we have used program variables, equality, logical variables (e.g., i), and conjunction ($\wedge$). What we allow in pre-conditions and post-conditions directly influences the sorts of program properties we can describe using partial correctness statements. We will use the set of logical formulas including comparisons

between arithmetic expressions, standard logical operators (and, or, implication, negation), as well as quantifiers (universal and existential). Assertions may also introduce logical variables that are different than the variables appearing in the program.

$$\begin{aligned}
i, j &\in \mathbf{LVar} \\
a &\in \mathbf{Aexp} ::= x \mid i \mid n \mid a_1 + a_2 \mid a_1 \times a_2 \\
P, Q &\in \mathbf{Assn} ::= \mathbf{true} \mid \mathbf{false} \mid a_1 < a_2 \mid P_1 \wedge P_2 \mid P_1 \vee P_2 \mid P_1 \Rightarrow P_2 \mid \neg P \mid \forall i. P \mid \exists i. P
\end{aligned}$$

Observe that the domain of boolean expressions $\mathbf{Bexp}$ is a subset of the domain of assertions. Notable additions over the syntax of boolean expression are quantifiers ($\forall$ and $\exists$). For instance, one can express the fact that variable x divides variable y using existential quantification: $\exists i. x \times i = y$.

3 Satisfaction and Validity

Now we would like to describe what we mean by “assertion P holds in store σ ”. But to determine whether P holds or not, we need more than just the store σ (which maps program variables to their values); we also need to know the values of the logical variables. We describe those values using an interpretation I ,

$$I : \mathbf{LVar} \rightarrow \mathbf{Int},$$

and define the function $\mathcal{A}_i[[a]]$, which is like the denotation of expressions extended to logical variables in the obvious way:

$$\begin{aligned}
\mathcal{A}_i[[n]](\sigma, I) &= n \\
\mathcal{A}_i[[x]](\sigma, I) &= \sigma(x) \\
\mathcal{A}_i[[i]](\sigma, I) &= I(i) \\
\mathcal{A}_i[[a_1 + a_2]](\sigma, I) &= \mathcal{A}_i[[a_1]](\sigma, I) + \mathcal{A}_i[[a_2]](\sigma, I)
\end{aligned}$$

Now we can express the satisfiability of assertions as a relation $\sigma \models_I P$ read as “ P is satisfied in store σ under interpretation I ,” or “store σ satisfies assertion P under interpretation I .” We will write $\sigma \not\models_I P$ whenever $\sigma \models_I P$ doesn’t hold.

$\sigma \models_I \mathbf{true}$	(always)
$\sigma \models_I a_1 < a_2$	if $\mathcal{A}_i[[a_1]](\sigma, I) < \mathcal{A}_i[[a_2]](\sigma, I)$
$\sigma \models_I P_1 \wedge P_2$	if $\sigma \models_I P_1$ and $\sigma \models_I P_2$
$\sigma \models_I P_1 \vee P_2$	if $\sigma \models_I P_1$ or $\sigma \models_I P_2$
$\sigma \models_I P_1 \Rightarrow P_2$	if $\sigma \not\models_I P_1$ or $\sigma \models_I P_2$
$\sigma \models_I \neg P$	if $\sigma \not\models_I P$
$\sigma \models_I \forall i. P$	if $\forall k \in \mathbf{Int}. \sigma \models_{I[i \mapsto k]} P$
$\sigma \models_I \exists i. P$	if $\exists k \in \mathbf{Int}. \sigma \models_{I[i \mapsto k]} P$

We can now say that an assertion P is valid (written $\models P$) if it is valid in any store, under any interpretation: $\forall \sigma, I. \sigma \models_I P$.

Having defined validity for individual assertions, we now turn to partial correctness statements. We say that a partial correctness statement $\{P\} c \{Q\}$ is satisfied in store σ and interpretation I , written $\sigma \models_I \{P\} c \{Q\}$, if:

$$\forall \sigma'. \text{ if } \sigma \models_I P \text{ and } C \llbracket c \rrbracket \sigma = \sigma' \text{ then } \sigma' \models_I Q$$

Note that this definition depends on the execution of c in the initial store σ .

Finally, we can say that a partial correctness triple is valid (written $\models \{P\} c \{Q\}$), if it is valid in any store and interpretation:

$$\forall \sigma, I. \sigma \models_I \{P\} c \{Q\}.$$

Now we know what we mean when we say “assertion P holds” or “partial correctness statement $\{P\} c \{Q\}$ is valid.”

4 Hoare Logic

How do we show that a partial correctness statement $\{P\} c \{Q\}$ holds? We know that $\{P\} c \{Q\}$ is valid if it holds for all stores and interpretations: $\forall \sigma, I. \sigma \models_I \{P\} c \{Q\}$. Furthermore, showing that $\sigma \models_I \{P\} c \{Q\}$ requires reasoning about the execution of command c (that is, $C \llbracket c \rrbracket$), as indicated by the definition of validity.

It turns out that there is an elegant way of deriving valid partial correctness statements, without having to reason about stores, interpretations, and the execution of c . We can use a set of inference rules and axioms, called *Hoare* rules, to directly derive valid partial correctness statements. The set of rules forms a proof system known as Hoare logic.

$$\begin{array}{c} \text{SKIP} \frac{}{\{P\} \text{ skip } \{P\}} \qquad \text{ASSIGN} \frac{}{\{P[a/x]\} x := a \{P\}} \\[10pt] \text{SEQ} \frac{\{P\} c_1 \{R\} \quad \{R\} c_2 \{Q\}}{\{P\} c_1; c_2 \{Q\}} \qquad \text{IF} \frac{\{P \wedge b\} c_1 \{Q\} \quad \{P \wedge \neg b\} c_2 \{Q\}}{\{P\} \text{ if } b \text{ then } c_1 \text{ else } c_2 \{Q\}} \\[10pt] \text{WHILE} \frac{\{P \wedge b\} c \{P\}}{\{P\} \text{ while } b \text{ do } c \{P \wedge \neg b\}} \end{array}$$

The assertion P in the rule for while loops is essentially a loop invariant; it is an assertion that holds before and after each iteration, as shown in the premise of the rule. Therefore, it is both a pre-condition for the loop (because it holds before the first iteration); and also a post-condition for the loop (because it holds after the last iteration). The fact that P is both a pre- and post-condition for the while loop is reflected in the conclusion of the rule.

There is one more rule, the rule of consequence, which allows to strengthen pre-conditions and weaken post-conditions:

$$\text{CONSEQUENCE} \frac{\models (P \Rightarrow P') \quad \{P'\} c \{Q'\} \quad \models (Q' \Rightarrow Q)}{\{P\} c \{Q\}}$$

These set of Hoare rules represent an inductive definition for a set of partial correctness statements $\{P\} c \{Q\}$. We will say that $\{P\} c \{Q\}$ is a theorem in Hoare logic, written $\vdash \{P\} c \{Q\}$, if we can build a finite proof tree for it.

5 Soundness and Completeness

At this point we have two kinds of partial correctness assertions:

- valid partial correctness statements $\models \{P\} c \{Q\}$, which hold for all stores and interpretations, according to the semantics of c , and
- Hoare logic theorems $\vdash \{P\} c \{Q\}$, that is, partial correctness statements that can be derived using the axioms and rules of Hoare logic.

The question is how do these sets relate to each other? More precisely, we have to answer two questions. First, is each Hoare logic theorem guaranteed to be valid partial correctness triple? In other words,

$$\text{does } \vdash \{P\} c \{Q\} \text{ imply } \models \{P\} c \{Q\}?$$

The answer is yes, and it shows that Hoare logic is sound. Soundness is important because it says that Hoare logic doesn't allow us to derive partial correctness assertions that actually don't hold. The proof of soundness requires induction on the derivations in $\vdash \{P\} c \{Q\}$ (we omit this proof).

The second question refers to the expressiveness and power of Hoare rules: can we always build a Hoare logic proof for each valid assertion? In other words,

$$\text{does } \models \{P\} c \{Q\} \text{ imply } \vdash \{P\} c \{Q\}?$$

The answer is a qualified yes: if $\models \{P\} c \{Q\}$ then there is a proof of $\vdash \{P\} c \{Q\}$ using the rules of Hoare logic, provided there are proofs for the validity of assertions that occur in the rule of consequence $\models (P \Rightarrow P')$ and $\models (Q' \Rightarrow Q)$. This result is known as the *relative completeness of Hoare logic* and is due to Cook (1974).

6 Example: Factorial

As an example illustrating how we can use Hoare logic to verify the correctness of a program, consider a program that computes the factorial of a number n :

```

{x = n ∧ n > 0}
y := 1;
while x > 0 do (
  y := y * x;
  x := x - 1
)
{y = n!}

```

Because the derivation for this proof is somewhat large, we will go through the reasoning used to construct it step by step.

At the top level, the program is a sequence of an assignment and a loop. To use the SEQ rule, we need to find an assertion that holds after the assignment and before the loop. Examining the rule for while loops, we see that the assertion before the loop must be an invariant for the loop. Inspecting the loop we see that it builds the factorial up in y starting with n , then multiplying it by $n - 1$, then $n - 2$, etc. At each iteration, x contains the next value multiplied into y , that is:

$$y = n * (n - 1) * \dots * (x + 1)$$

If we multiply both sides of this equality by $x!$ and re-write the equality we get $x! * y = n!$, which is an invariant for the loop. However, to make the proof go through, we need a slightly stronger invariant:

$$I = x! * y = n! \wedge x \geq 0$$

Having identified a suitable loop invariant, let us take a step back and review where we are. We want to prove that our overall partial correctness specification is valid. To do this, we need to show two facts:

$$\{x = n \wedge n > 0\} y := 1 \{I\} \tag{1}$$

$$\{I\} \textbf{while } x > 0 \textbf{ do } (y := y * x; x := x - 1) \{y = n!\} \tag{2}$$

After showing that both (1) and (2) hold, we can use the rule SEQ to obtain the desired result.

To show (1), we use the ASSIGN axiom and obtain the following: $\{I[1/y]\} y := 1 \{I\}$. Expanding this out, we obtain:

$$\{x! * 1 = n! \wedge x \geq 0\} y := 1 \{x! * y = n! \wedge x \geq 0\}$$

With the following implication,

$$x = n \wedge n > 0 \implies x! * 1 = n! \wedge x \geq 0,$$

(which can be shown by an easy calculation) we obtain (1) using the rule CONSEQUENCE.

Now let us prove (2). To use the WHILE rule, we need to show that I is an invariant for the loop:

$$\{I \wedge x > 0\} y := y * x; x := x - 1 \{I\} \tag{3}$$

We will show this by going backwards through the sequence of assignments:

$$\{(x - 1)! * y = n! \wedge (x - 1) \geq 0\} x := x - 1 \{I\} \tag{4}$$

$$\{(x - 1)! * y * x = n! \wedge (x - 1) \geq 0\} y := y * x \{(x - 1)! * y = n! \wedge (x - 1) \geq 0\} \tag{5}$$

Then, using the following implication:

$$I \wedge x > 0 \implies (x - 1)! * y * x = n! \wedge (x - 1) \geq 0$$

we obtain (3) using CONSEQUENCE, (4), and (5). Thus, I is an invariant for the loop and so by WHILE we obtain,

$$\{I\} \textbf{while } x > 0 \textbf{ do } (y := y * x; x := x - 1) \{I \wedge x \leq 0\}$$

To finish the proof, we just have to show

$$\begin{aligned} I \wedge x \leq 0 &\implies y = n! \\ \text{i.e., } x! * y = n! \wedge x \geq 0 \wedge x \leq 0 &\implies y = n! \end{aligned}$$

which holds as $x \geq 0$ and $x \leq 0$ implies $x = 0$ and so $x! = 1$. The result follows by CONSEQUENCE.